In God we trust

نام و نام خانوادگی : مهدیس میرزایی

شماره دانشجویی : ۸۱۰۱۰۲۵۵۰

شماره پروژه : پروژه سوم

بخش یک :

۱ - در این قسمت صرفا mapset را ایجاد می‌کنیم.

```
symbol_list = ['a':'z' ' ' ',' ',' '!' ';'];
binary_codes = cell(1, length(symbol_list));

for k = 1:length(symbol_list)
    binary_codes{k} = dec2bin(k-1, 5);
end

Mapset = cell(2, length(symbol_list));
for k = 1:length(symbol_list)
    Mapset{1,k} = symbol_list(k);
    Mapset{2,k} = binary_codes{k};
end
disp("Mapset:");
disp(Mapset(:, 1:8));
```

و خروجی به صورت زیر است.

```
Mapset:
    {'a'    }    {'b'    }    {'c'    }    {'d'    }    {'e'    }    {'f'    }    {'g'    }    {'h'    }
    {'00000'}    {'00001'}    {'00010'}    {'00011'}    {'00100'}    {'00101'}    {'00110'}    {'00111'}
```

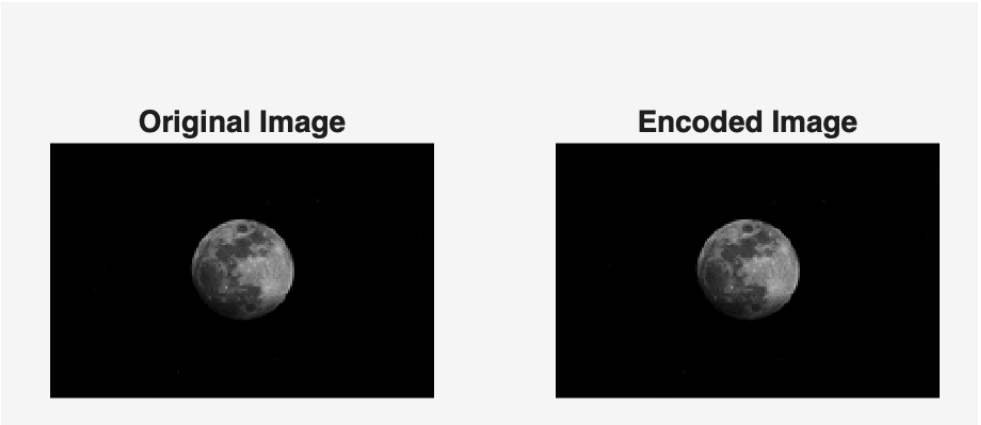
۲- تابع coding را به صورت زیر پیاده‌سازی کنیم. در این تابع تصویر را به بلوک‌های ۵×۵ تقسیم می‌کند و فقط در بلوک‌هایی که انحراف معیار آن (یعنی تغییر شدت روشنایی) از آستانه‌ی مشخص بیشتر است، این بیت‌ها را در بیت کم‌اهمیت (LSB) پیکسل‌ها قرار می‌دهد و این باعث تغییر در نواحی پربافت تصویر پنهان می‌شود تا تغییرات قابل‌دیدن نباشند.

```
function coded_img = coding(msg, Mapset, img, th)
    msg = [msg ';'];
    bin_msg = '';
    for k = 1:length(msg)
        found = false;
        for idx = 1:size(Mapset, 2)
            if Mapset{1,idx} == msg(k)
                bin_msg = [bin_msg Mapset{2,idx}];
                found = true;
                break;
            end
        end
        if ~found
            error('Character "%c" not found in Mapset.', msg(k));
        end
    end
    if length(bin_msg) > numel(img)
        error('Message too long for selected image.');
    end
    coded_img = uint8(img);
    [rows, cols] = size(img);
    blk_size = 5;
    bit_idx = 1;
    bin_msg_num = bin_msg - '0';
    for r = 1:blk_size:rows-blk_size+1
        for c = 1:blk_size:cols-blk_size+1
            block = img(r:r+blk_size-1, c:c+blk_size-1);
            if std(double(block(:))) > th
                flat = block(:);
                for k = 1:length(flat)
                    if bit_idx > length(bin_msg_num)
                        break;
                    end
                    flat(k) = bitset(flat(k), 1, bin_msg_num(bit_idx));
                    bit_idx = bit_idx + 1;
                end
                coded_img(r:r+blk_size-1, c:c+blk_size-1) = reshape(flat, blk_size, blk_size);
            end
            if bit_idx > length(bin_msg_num)
                break;
            end
        end
        if bit_idx > length(bin_msg_num)
            break;
        end
    end
    if bit_idx <= length(bin_msg_num)
        error('Not enough textured blocks to hide entire message.');
    end
end
```

۳- تابع decoding در واقع کاری برعکس coding را انجام می‌دهد. این تابع پیام پنهان‌شده را از تصویر بازیابی می‌کند. مجدد تصویر را به بلوک‌های ۵×۵ تقسیم می‌کند و از بلوک‌هایی که واریانس‌شان از مقدار آستانه بیشتر است، بیت کم‌اهمیت (LSB) هر پیکسل را استخراج می‌کند و با کنار هم گذاشتن گروه‌های ۵تایی در mapset مقایسه می‌کند و کاراکتر را به دست می‌آوریم و کاراکتر ; به معنی پایان پیام است.

```
function msg = decoding(coded_img, Mapset, th)
    coded_img = uint8(coded_img);
    blk_size = 5;
    [rows, cols] = size(coded_img);
    bits = [];
    for r = 1:blk_size:rows-blk_size+1
        for c = 1:blk_size:cols-blk_size+1
            block = coded_img(r:r+blk_size-1, c:c+blk_size-1);
            if std(double(block(:))) > th
                flat = block(:);
                bits = [bits; bitget(flat, 1)];
            end
        end
    end
    n = floor(length(bits)/5);
    if n == 0
        msg = '';
        return;
    end
    bits = bits(1:n*5);
    bits5 = reshape(bits, 5, [])';
    msg = '';
    for i = 1:n
        bin_str = sprintf('%d', bits5(i,:));
        for idx = 1:size(Mapset, 2)
            if strcmp(Mapset{2,idx}, bin_str)
                c = Mapset{1,idx};
                if c == ';'
                    return;
                end
                msg = [msg c];
                break;
            end
        end
    end
end
```

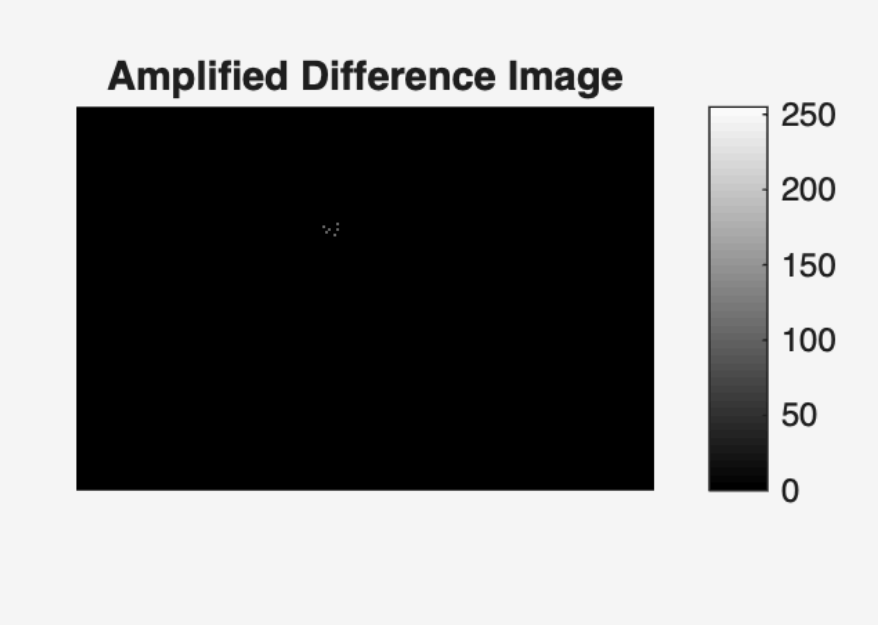
۴- مراحل ورودی گرفتن عکس و سیاه و سفید کردن آن و در نهایت آن را به تابع coding می‌دهیم و عکس‌ها را نمایش می‌دهیم . حال با استفاده از تابع decode عمل برعکس انجام می‌دهیم و برای اینکه مطمئن باشیم درست عمل کرده است خروجی‌ها را بررسی می‌کنیم و مشاهده می‌شود هردو moon است.




```
[file_name, file_path] = uigetfile({'*.png;*.jpg;*.bmp','Image Files'}, 'Select Image');
if isequal(file_name,0)
    error('No image selected.');
end

input_img = imread(fullfile(file_path, file_name));
encoded_img = coding('moon;', Mapset, input_img, 30);
```

و در نهایت در این قسمت میزان اختلاف عددی بین پیکسل‌های دو تصویر محاسبه شده و نمایش داده شده است.



و طبق خروجی در تصویر متن moon پنهان شده بود.

```
Decoded message: moon
>>
```

بخش دو :

برای این قسمت از کدی که در کلاس زده شده است استفاده کردیم. در این فرایند هدف پیدا کردن اشیا متحرک است پس وقتی که اشیا خارجی داشته باشیم که هواپیما نباشد (مثل دکل‌ها) باید به صورت دستی حذف شود پس کاربر آنها را انتخاب می‌کند و با زدن enter و سپس سوالی که پرسیده می‌شود که آیا باز هم چیزی می‌خواهد انتخاب شود یا خیر.

```
clc; clear; close all;
videoFile = '1-1.mp4';
vid = VideoReader(videoFile);

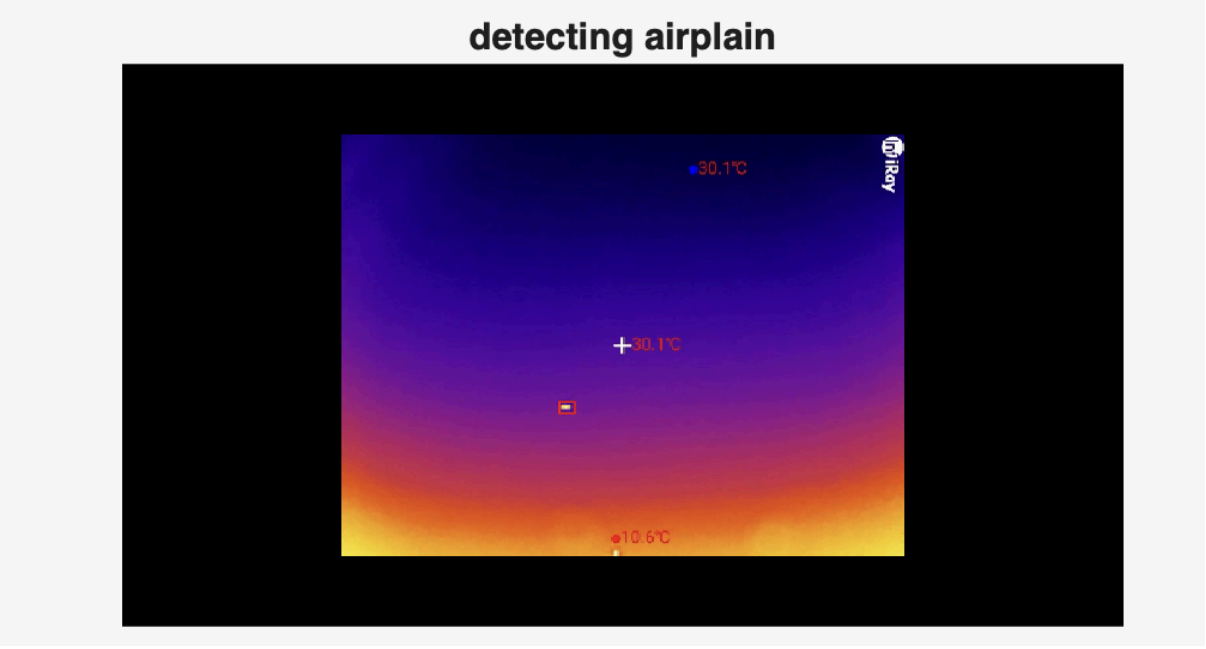
outputFile = 'target_tracking.mp4';
outputVideo = VideoWriter(outputFile);
outputVideo.FrameRate = vid.FrameRate;
open(outputVideo);

firstFrame = readFrame(vid);
firstGray = rgb2gray(firstFrame);
figure; imshow(firstFrame);
title('select, after that enter');

mask = false(size(firstGray));
hold on;
rects = {};
while true
    h = drawrectangle('Color', 'cyan');
    wait(h);
    rect = round(h.Position);
    if isempty(rect)
        break;
    end
    rects{end+1} = rect;
    mask(rect(2):(rect(2)+rect(4)), rect(1):(rect(1)+rect(3))) = true;
    answer = questdlg('Continue?', ...
    'Region Selection', 'Yes', 'No', 'No');

    if strcmp(answer, 'No')
        break;
    end
end
hold off;
close;
```

سپس پس از حذف تمامی آن نواحی قسمت‌هایی که تغییر میکند فریم به فریم شناسایی می‌شود و برای آنها کادر رسم می‌شود.



```
figure; imshow(mask);
frame1_gray = firstGray;
frame1_gray(mask) = 0;
frame1_gray = mat2gray(frame1_gray);

while hasFrame(vid)
    frame2 = readFrame(vid);
    frame2_gray = rgb2gray(frame2);
    frame2_gray(mask) = 0;
    frame2_gray = mat2gray(frame2_gray);
    diffFrame = imabsdiff(frame2_gray, frame1_gray);

    bw = diffFrame > 0.05;
    bw = imgaussfilt(double(bw), 2) > 0.1;
    bw = bwareaopen(bw, 5);

    stats = regionprops(bw, 'BoundingBox', 'Centroid');

    frameOut = frame2;

    for k = 1:length(stats)
        bbox = stats(k).BoundingBox;
        x = round(bbox(1)); y = round(bbox(2)); w = round(bbox(3)); h = round(bbox(4));
        red = [255, 0, 0];
        frameOut(y:y+1, x:x+w, 1) = red(1);
        frameOut(y:y+1, x:x+w, 2) = red(2);
        frameOut(y:y+1, x:x+w, 3) = red(3);
        frameOut(y+h-1:y+h, x:x+w, 1) = red(1);
        frameOut(y+h-1:y+h, x:x+w, 2) = red(2);
        frameOut(y+h-1:y+h, x:x+w, 3) = red(3);
        frameOut(y:y+h, x:x+1, 1) = red(1);
        frameOut(y:y+h, x:x+1, 2) = red(2);
        frameOut(y:y+h, x:x+1, 3) = red(3);
        frameOut(y:y+h, x+w-1:x+w, 1) = red(1);
        frameOut(y:y+h, x+w-1:x+w, 2) = red(2);
        frameOut(y:y+h, x+w-1:x+w, 3) = red(3);
        centroid = round(stats(k).Centroid);
        cx = centroid(1); cy = centroid(2);
        yellow = [255, 255, 0];
        frameOut(cy-5:cy+5, cx, 1) = yellow(1);
        frameOut(cy-5:cy+5, cx, 2) = yellow(2);
        frameOut(cy-5:cy+5, cx, 3) = yellow(3);
        frameOut(cy, cx-5:cx+5, 1) = yellow(1);
        frameOut(cy, cx-5:cx+5, 2) = yellow(2);
        frameOut(cy, cx-5:cx+5, 3) = yellow(3);
    end
    imshow(frameOut);
    title('detecting airplain');
    drawnow;
    writeVideo(outputVideo, frameOut);
    frame1_gray = frame2_gray;
end
disp('end of detecting');
close(outputVideo);
disp(['output file: ', outputFile]);
```

و در نهایت این فریم‌ها در فایلی به صورت ویدیو ذخیره می‌شود.